# UT2. PROGRAMACIÓN MULTIHILO

## ACTIVIDAD 2

## Creación de múltiples hilos

**Título**: Ejecución concurrente de múltiples hilos en Java.

**Descripción**:

Vas a crear 8 hilos que se ejecuten de forma concurrente usando los dos mecanismos que ofrece Java: **extender la clase Thread** e **implementar la interfaz Runnable**. Cada hilo mostrará su nombre y su identificador, y comprobarás que el orden de ejecución lo decide la JVM, no tu programa.

**Objetivos**:

- Crear e iniciar hilos con Thread y con Runnable.
- Comprobar que los hilos se ejecutan de forma concurrente y en un orden no predecible.
- Diferenciar start() de run() y Thread de Runnable.

**Instrucciones:**

Cada apartado se programa en un único archivo .java con dos clases. Solo una de ellas puede ser public, y esa clase da nombre al archivo.

1. **Creación de 8 hilos extendiendo la clase Thread** (archivo MultihiloThread.java):

   - Crea la clase MultihiloThreadDemo, que extienda Thread. Sobrescribe el método run() con @Override para mostrar el nombre y el ID del hilo en ejecución con Thread.currentThread().getName() y Thread.currentThread().threadId(). No uses getId(), que está obsoleto desde Java 19.
   - Crea la clase principal MultihiloThread. Lo primero que debe hacer main es mostrar tu nombre y el ID del hilo main. Después, con un bucle, crea 8 objetos MultihiloThreadDemo e inícialos con start().
   - Ejecuta el programa varias veces y compara las salidas.
   - Haz el experimento: cambia start() por run(), ejecuta y observa qué ocurre. Después vuelve a dejar start().

Tienes esta estructura inicial. Completa donde indica **'TODO'**:

```
class MultihiloThreadDemo extends Thread {

    @Override
    public void run() {
        // TODO: muestra el nombre y el ID del hilo que está ejecutando este código.
        //       Pista: Thread.currentThread() devuelve ese hilo.
    }
}

public class MultihiloThread {

    public static void main(String[] args) {
```

```
        int numHilos = 8;

        // TODO: muestra tu nombre y el ID del hilo main.

        for (int i = 0; i < numHilos; i++) {
            // TODO: crea un objeto MultihiloThreadDemo y arráncalo con start().
        }
    }
}
```

2. **Creación de 8 hilos implementando la interfaz Runnable** (archivo MultihiloRunnable.java)**:**
   - Crea la clase MultihiloRunnableDemo, que implemente Runnable, con un método run() que muestre lo mismo que en el apartado 1.
   - Crea la clase principal MultihiloRunnable. En main, muestra tu nombre y el ID del hilo main. Después crea **un único** objeto MultihiloRunnableDemo y, con un bucle, crea 8 objetos Thread pasándole ese mismo objeto al constructor. Inicia cada hilo con start().

**Ten en cuenta:** los ID no empiezan en 0. La JVM crea antes sus propios hilos internos, así que es normal que veas valores como 18, 19… El orden de los mensajes puede cambiar en cada ejecución, y eso no es un error.

**Preguntas** (responde en 2 o 3 líneas cada una):

1. En el experimento del apartado 1, ¿qué nombre e ID mostraron los mensajes al usar run() en lugar de start()? ¿Por qué?

   Mostraron el nombre main y el ID del hilo principal. Esto ocurre porque llamar a run() directamente ejecuta el método como si fuera un método normal y secuencial dentro del hilo principal, sin crear un hilo nuevo activado.

2. ¿Por qué el orden de los mensajes cambia entre ejecuciones?

   Porque la planificación de los hilos depende exclusivamente de la JVM y del sistema operativo. Al ejecutarse de forma concurrente, el sistema decide qué hilo entra a la CPU.

3. ¿Qué ventaja tiene implementar Runnable frente a extender Thread?

Implementa mayor flexibilidad ya que en Java no existe la herencia múltiple; si una clase implementa Runnable, todavía puede heredar de otra clase, además de separar la lógica de la tarea del mecanismo de control del hilo.

# Rúbrica:

| | |
|---|---|
| **0** | No entrega, o el código no compila. |
| **3** | Solo funciona uno de los dos apartados (Thread o Runnable). |
| **5** | Funcionan los dos apartados, pero faltan las capturas o las preguntas. |
| **7** | Funcionan los dos apartados y las capturas están completas. Las preguntas están |

| | |
|---|---|
| | incompletas o tienen errores. |
| **9** | Todo correcto: código, capturas y preguntas razonadas. |
| **10** | Todo correcto y, además, incluye una mejora o un detalle que aporta valor al programa. |

# Entrega:

i. Incluye aquí el código fuente completo de **MultihiloThread.java** (clases **MultihiloThreadDemo y MultihiloThread)**:

**MultihiloThreadDemo** :

```
/**
 * @author Juan Gabriel Galarza Claros
 */
class MultihiloThreadDemo extends Thread {
    @Override
    public void run() {
        // Muestra el nombre y el ID del hilo en ejecución
        System.out.println("Hilo ejecutándose -> Nombre: " + Thread.currentThread().getName() + ", ID: "
+ Thread.currentThread().threadId());
    }
}
```

**MultihiloThread** :

```
public class MultihiloThread {
    public static void main(String[] args) {
        int numHilos = 8;

        // Muestra el nombre y el ID del hilo main,reemplaza TuNombre por tu nombre real
        System.out.println("\n Hilo Principal (main) -> Nombre: TuNombre, ID: " +
Thread.currentThread().threadId());

        for (int i = 0; i < numHilos; i++) {
            // Crea un objeto MultihiloThreadDemo y arráncalo con start()
            MultihiloThreadDemo hilo = new MultihiloThreadDemo();
            hilo.start();
        }
    }
}
```

ii. Incluye aquí el código fuente completo de **MultihiloRunnable.java** (clases **MultihiloRunnableDemo** y **MultihiloRunnable**):

**MultihiloRunnableDemo :**

```
class MultihiloRunnableDemo implements Runnable {
    @Override
    public void run() {
        // Muestra el nombre y el ID del hilo en ejecución
        System.out.println("Hilo Runnable -> Nombre: " + Thread.currentThread().getName() + ", ID: " +
Thread.currentThread().threadId());
```

```
    }
}
```

**MultihiloRunnable :**

```
class MultihiloRunnable {
    public static void main(String[] args) {
        int numHilos = 8;

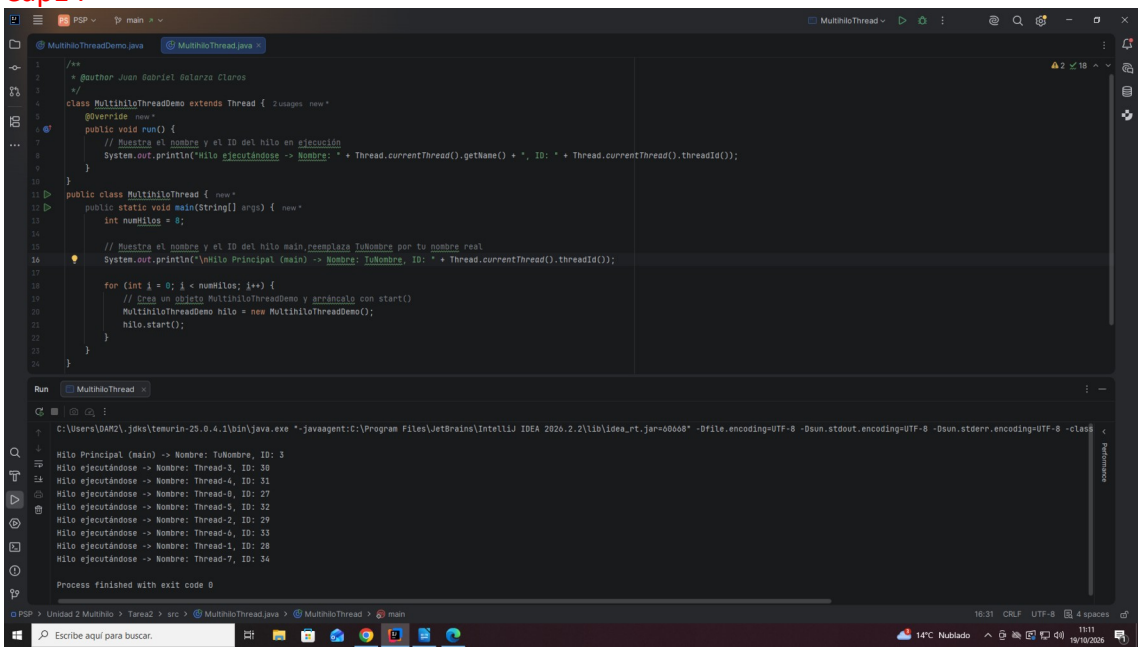
        // Muestra el nombre y el ID del hilo main
        System.out.println("Hilo Principal (main) -> Nombre: TuNombre, ID: " +
Thread.currentThread().threadId());

        // Crea un único objeto de la clase que implementa Runnable
        MultihiloRunnableDemo tarea = new MultihiloRunnableDemo();

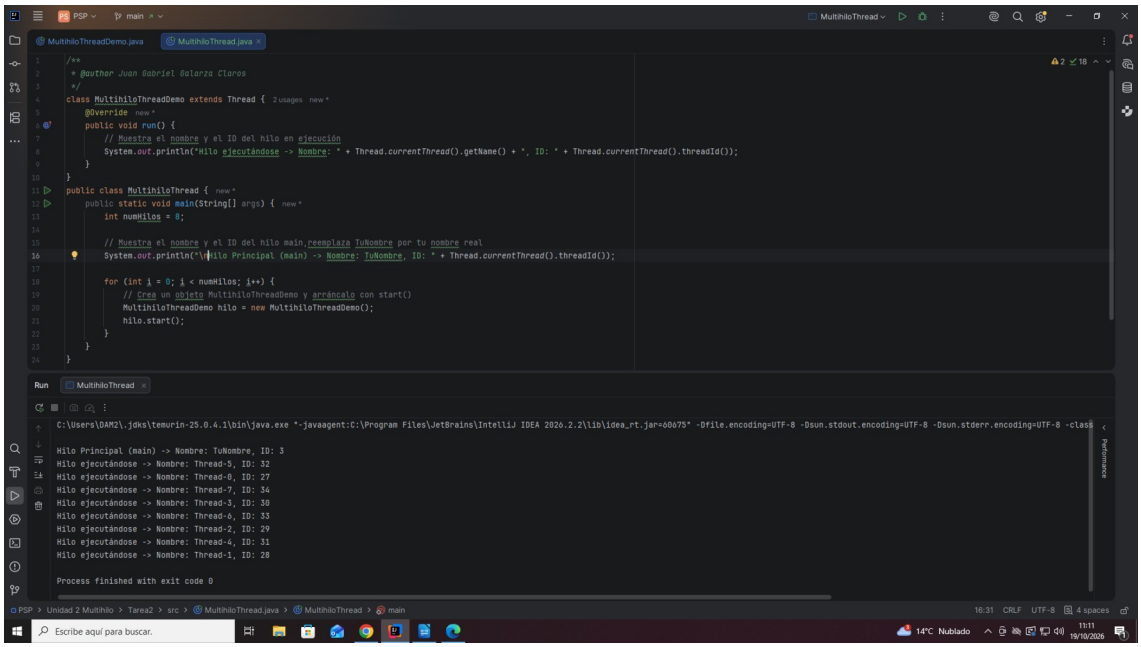
        for (int i = 0; i < numHilos; i++) {
            // Crea un objeto Thread pasando el objeto Runnable y arráncalo con start()
            Thread hilo = new Thread(tarea);
            hilo.start();
        }
    }
}
```

iii. Incluye aquí 3 capturas de pantalla: dos ejecuciones de MultihiloThread en las que cambie el orden de los mensajes y una ejecución de MultihiloRunnable. En todas debe verse tu nombre.

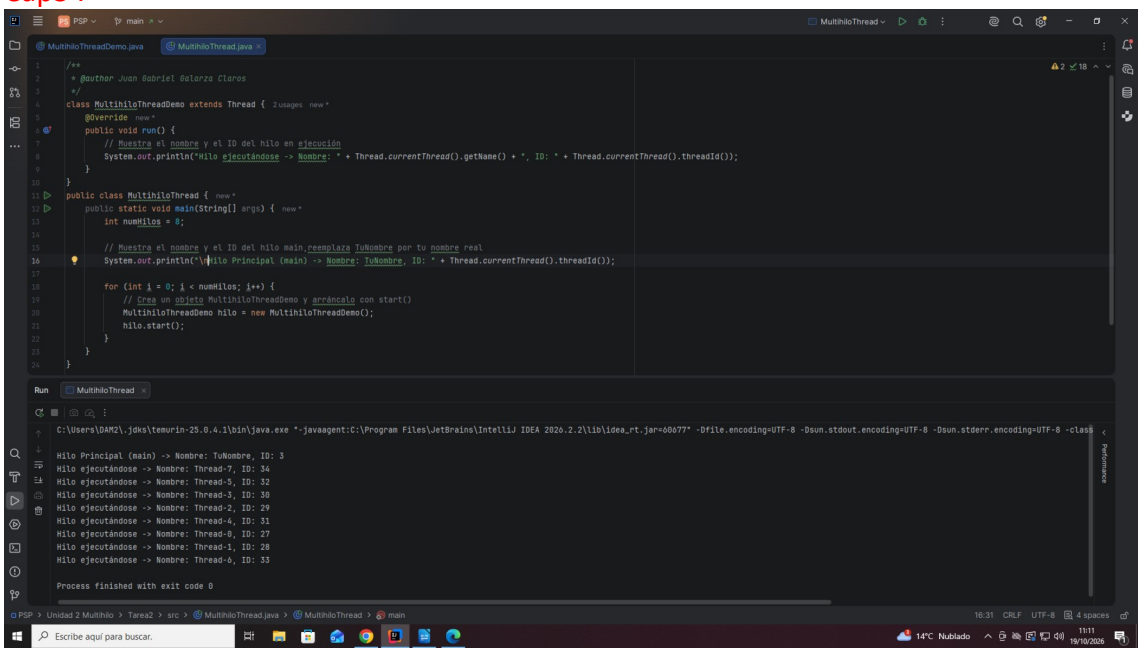
Cap1 :




Cap2 :


/**
* @author Juan Gabriel Galarza Claros
*/
class MultihiloThreadDemo extends Thread {
@Override
public void run() {
// Muestra el nombre y el ID del hilo en ejecución
System.out.println("Hilo ejecutándose -> Nombre: " + Thread.currentThread().getName() + ", ID: " + Thread.currentThread().threadId());
}
}
public class MultihiloThread {
public static void main(String[] args) {
int numHilos = 8;
// Muestra el nombre y el ID del hilo main,reemplaza TuNombre por tu nombre real
System.out.println("\nHilo Principal (main) -> Nombre: TuNombre, ID: " + Thread.currentThread().threadId());
for (int i = 0; i < numHilos; i++) {
// Crea un objeto MultihiloThreadDemo y arráncalo con start()
MultihiloThreadDemo hilo = new MultihiloThreadDemo();
hilo.start();
}
}
}
Run
MultihiloThread
C:\Users\DAM2\.jdks\temurin-25.0.4.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.2\lib\idea_rt.jar=60675" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -class
Hilo Principal (main) -> Nombre: TuNombre, ID: 3
Hilo ejecutándose -> Nombre: Thread-5, ID: 32
Hilo ejecutándose -> Nombre: Thread-0, ID: 27
Hilo ejecutándose -> Nombre: Thread-7, ID: 34
Hilo ejecutándose -> Nombre: Thread-3, ID: 30
Hilo ejecutándose -> Nombre: Thread-6, ID: 33
Hilo ejecutándose -> Nombre: Thread-2, ID: 29
Hilo ejecutándose -> Nombre: Thread-4, ID: 31
Hilo ejecutándose -> Nombre: Thread-1, ID: 28
Process finished with exit code 0

Cap3 :


/**
* @author Juan Gabriel Galarza Claros
*/
class MultihiloThreadDemo extends Thread {
@Override
public void run() {
// Muestra el nombre y el ID del hilo en ejecución
System.out.println("Hilo ejecutándose -> Nombre: " + Thread.currentThread().getName() + ", ID: " + Thread.currentThread().threadId());
}
}
public class MultihiloThread {
public static void main(String[] args) {
int numHilos = 8;
// Muestra el nombre y el ID del hilo main,reemplaza TuNombre por tu nombre real
System.out.println("\nHilo Principal (main) -> Nombre: TuNombre, ID: " + Thread.currentThread().threadId());
for (int i = 0; i < numHilos; i++) {
// Crea un objeto MultihiloThreadDemo y arráncalo con start()
MultihiloThreadDemo hilo = new MultihiloThreadDemo();
hilo.start();
}
}
}
Run
MultihiloThread
C:\Users\DAM2\.jdks\temurin-25.0.4.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.2\lib\idea_rt.jar=60677" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -class
Hilo Principal (main) -> Nombre: TuNombre, ID: 3
Hilo ejecutándose -> Nombre: Thread-7, ID: 34
Hilo ejecutándose -> Nombre: Thread-5, ID: 32
Hilo ejecutándose -> Nombre: Thread-3, ID: 30
Hilo ejecutándose -> Nombre: Thread-2, ID: 29
Hilo ejecutándose -> Nombre: Thread-4, ID: 31
Hilo ejecutándose -> Nombre: Thread-0, ID: 27
Hilo ejecutándose -> Nombre: Thread-1, ID: 28
Hilo ejecutándose -> Nombre: Thread-6, ID: 33
Process finished with exit code 0